```
        var tokenId = delegate.create(request, token);    #A
        return SecretBox.encrypt(encryptionKey,
tokenId).toString();    #B
    }

    @Override
    public Optional<Token> read(Request request, String
tokenId) {
        var box = SecretBox.fromString(tokenId);    #C
        var originalTokenId =
box.decryptToString(encryptionKey);    #C
        return delegate.read(request, originalTokenId);    #C
    }

    @Override
    public void revoke(Request request, String tokenId) {
        var box = SecretBox.fromString(tokenId);    #C
        var originalTokenId =
box.decryptToString(encryptionKey);    #C
        delegate.revoke(request, originalTokenId);    #C
    }
}
```

#A Call the delegate TokenStore to generate the token ID.
#B Use the SecretBox.encrypt() method to encrypt the token.
#C Decode and decrypt the box and then use the original token ID.

As you can see, the `EncryptedTokenStore` using `SecretBox` is very short as the library takes care of almost all details for you. To use the new store, you'll need to generate a new key to use for encryption rather than reusing the existing HMAC key.

> **PRINCIPLE** A cryptographic key should only be used for a single purpose. Use separate keys for different